Modern Reinforcement Learning: From Agents to LLMs

Imitation Learning & Offline Reinforcement Learning

Lecture 6

Keywords: Imitation Learning, Behavior Cloning, DAgger, Offline Reinforcement Learning, Distribution Shift, Support Mismatch

Ilija Bogunovic

Today's Roadmap

1) Motivation & Recap – Acting vs learning from data
   Why fixed datasets change RL

2) Imitation Learning – Behavior Cloning from demonstrations
   Compounding error & distribution shift

3) Interactive Imitation (DAgger) – Learning on visited states
   Reducing error accumulation

4) From Imitation to Offline RL – Beyond copying behavior
   Why rewards matter again

5) Offline RL – Setup & failure modes
   Distribution shift & support mismatch

6) Conservative Learning – Limits of offline improvement
   Staying close to the data

📘 **Side note / "Keep in mind" box:**
- Distribution shift is the central challenge across imitation and offline RL.

Motivation & Recap: Acting vs Learning from Data

Two ways to learn policies

- **Learning by acting**
  - Agent interacts with the environment
  - Policy determines future data
  - Exploration is possible
- **Learning from data**
  - Dataset is fixed
  - No control over data distribution
  - No exploration / No correction of mistakes
- **Key question:** What changes when the agent cannot collect new data?

Where Does This Situation Arise?

- **Offline learning: common scenarios**
  - Expert demonstrations
  - Human-generated trajectories
  - Logged data from deployed systems
  - Safety-critical environments

- **Examples**
  - Robotics & autonomous driving
  - Recommendation & control systems
  - Language models trained from human data

Imitation Learning & Behavior Cloning

Learning from Demonstrations

Imitation learning

- Learn a policy from expert trajectories
- No explicit reward function
- No environment interaction

Given

- Dataset of demonstrations $D = \{(s_t, a_t)\}_t$

Goal

- Learn a policy that behaves like the expert

Behavior Cloning

- **Behavior Cloning (BC)**
  - Reduce imitation learning to supervised learning
  - Predict expert actions from states
- **Objective**

Why Behavior Cloning Is Appealing

- **Why BC is attractive**
  - Simple supervised learning objective
  - Stable and easy to optimize
  - Scales to large datasets
- **In practice**
  - Often the first step before RL
  - Widely used in real systems

The Hidden Assumption Behind Behavior Cloning

- Implicit assumption in BC
  - Training and test state distributions are similar
- In reality
  - Training states come from the expert
  - Test states come from the learned policy
- Consequence
  - Small errors change the state distribution

📘 Side note / “Keep in mind” box:
• This is a sequential version of classic covariate shift
• Unlike standard supervised learning, the learner creates its own inputs

Compounding Error in Behavior Cloning (Setup & Intuition)

- Policies and distributions
  - Expert policy: π_E
  - Learned policy (BC): π
  - State distribution at time t: $d_t^{\pi}(s) := \Pr(s_t = s \mid \pi)$

Compounding Error in Behavior Cloning (Setup & Intuition)

- Policies and distributions
  - Expert policy: π_E
  - Learned policy (BC): π
  - State distribution at time t: $d_t^{\pi}(s) := \Pr(s_t = s \mid \pi)$
- Training vs test distributions
  - Training data drawn from $d_t^{\pi_E}$
  - Test-time states drawn from d_t^{π}
- One-step prediction guarantee

$$\mathbb{P}_{s \sim d_t^{\pi_E}}\left[\pi(s) \neq \pi_E(s)\right] \leq \epsilon$$

Compounding Error in Behavior Cloning (Setup & Intuition)

- Policies and distributions
  - Expert policy: π_E
  - Learned policy (BC): π
  - State distribution at time t: $d_t^\pi(s) := \Pr(s_t = s \mid \pi)$
- Training vs test distributions
  - Training data drawn from $d_t^{\pi_E}$
  - Test-time states drawn from d_t^π
- One-step prediction guarantee

$$\mathbb{P}_{s \sim d_t^{\pi_E}}\left[\pi(s) \neq \pi_E(s)\right] \leq \epsilon$$

- **Key issue**
  - No guarantee on error under d_t^π
  - Small action errors change future state distributions
- **Core question:** How does a per-step error ϵ translate into total error over horizon T ?

Behavior Cloning: Error Compounding

- **Setup**
  - Expert policy: π_E
  - Learned policy: π
  - Horizon: T
- **Key assumption (under expert data)**

$$\mathbb{P}_{s \sim d_t^{\pi_E}}\left[\pi(s) \neq \pi_E(s)\right] \leq \epsilon \qquad \text{(Small one-step error)}$$

Behavior Cloning: Error Compounding

- **Setup**
  - Expert policy: π_E
  - Learned policy: π
  - Horizon: T
- **Key assumption (under expert data)**

$$\mathbb{P}_{s \sim d_t^{\pi_E}}\left[\pi(s) \neq \pi_E(s)\right] \leq \epsilon \qquad \text{(Small one-step error)}$$

- **What goes wrong?**
  - Probability of first mistake by time t: $\leq t\epsilon$ => Errors accumulate over time

Behavior Cloning: Error Compounding

- **Setup**
  - Expert policy: π_E
  - Learned policy: π
  - Horizon: T
- **Key assumption (under expert data)**

$$\mathbb{P}_{s \sim d_t^{\pi_E}}\left[\pi(s) \neq \pi_E(s)\right] \leq \epsilon \qquad \text{(Small one-step error)}$$

- **What goes wrong?**
  - Probability of first mistake by time t: $\leq t\epsilon$ => Errors accumulate over time
- **Total expected errors**

$$\mathbb{E}[\#\text{errors}] \leq \sum_{t=1}^{T} t\epsilon = \frac{T(T+1)}{2}\epsilon = O(T^2\epsilon)$$

Behavior Cloning: Error Compounding

- **Setup**
  - Expert policy: π_E
  - Learned policy: π
  - Horizon: T
- **Key assumption (under expert data)**

$$\mathbb{P}_{s\sim d_t^{\pi_E}}\left[\pi(s) \neq \pi_E(s)\right] \leq \epsilon \qquad \text{(Small one-step error)}$$

- **What goes wrong?**
  - Probability of first mistake by time t: $\leq t\epsilon$ => Errors accumulate over time
- **Total expected errors**

$$\mathbb{E}[\#\text{errors}] \leq \sum_{t=1}^{T} t\epsilon = \frac{T(T+1)}{2}\epsilon = O(T^2\epsilon)$$

- **Takeaway**
  - ❗ Errors grow quadratically with horizon
  - ❗ Caused by distribution shift (off-expert states)
  - ❗ Even tiny one-step error is not enough

📘 **Intuition (one line): A single mistake puts the agent in unseen states → more mistakes → cascade**

From Errors to Performance

Measure performance with loss

- At each step, assign a cost: $\ell(s, a) \in [0,1]$: “How bad was the action?”

Total loss over a trajectory $\sum_{t=1}^{T} \ell(s_t, \pi(s_t))$ (Total mistakes / cost over time)

From Errors to Performance

Measure performance with loss

- At each step, assign a cost: $\ell(s, a) \in [0,1]$: “How bad was the action?”

Total loss over a trajectory $\sum_{t=1}^{T} \ell(s_t, \pi(s_t))$ (Total mistakes / cost over time)

Key connection: Each mistake contributes at most 1 = > Total loss ≈ number of errors

From Errors to Performance

Measure performance with loss

- At each step, assign a cost: $\ell(s, a) \in [0,1]$: "How bad was the action?"

Total loss over a trajectory $\sum_{t=1}^{T} \ell(s_t, \pi(s_t))$ (Total mistakes / cost over time)

Key connection: Each mistake contributes at most 1 = > Total loss ≈ number of errors

Result

$$\text{BC performance} \leq \text{Expert performance} + O(T^2\epsilon)$$

From Errors to Performance

Measure performance with loss

- At each step, assign a cost: $\ell(s, a) \in [0,1]$: "How bad was the action?"

Total loss over a trajectory $\sum_{t=1}^{T} \ell(s_t, \pi(s_t))$ (Total mistakes / cost over time)

Key connection: Each mistake contributes at most 1 = > Total loss ≈ number of errors

Result

$$\text{BC performance} \leq \text{Expert performance} + O(T^2\epsilon)$$

Takeaway

- Small one-step error ⇒ large total loss
- Errors compound over time (distribution shift)

From Errors to Performance

Measure performance with loss

- At each step, assign a cost: $\ell(s, a) \in [0,1]$: "How bad was the action?"

Total loss over a trajectory $\sum_{t=1}^{T} \ell(s_t, \pi(s_t))$ (Total mistakes / cost over time)

Key connection: Each mistake contributes at most 1 = > Total loss ≈ number of errors

Result

$$\text{BC performance} \leq \text{Expert performance} + O(T^2\epsilon)$$

Takeaway

- Small one-step error ⇒ large total loss
- Errors compound over time (distribution shift)

Why this matters: Training only on expert states is not enough

- Need to train on states visited by the learned policy: This motivates DAgger

DAgger: Core Idea

Problem in BC

- Training distribution: $d_t^{\pi_E}$
- Test distribution: d_t^{π}
- Error bound scales as $O(T^2\epsilon)$

Key idea (DAgger)

Train on the state distribution induced by the learned policy

Principle

- Iteratively collect data from d_t^{π}
- Query expert labels on visited states
- Aggregate datasets to eliminate distribution shift

DAgger Algorithm

Inputs:

- Expert policy π_E
- Initial dataset D_0 (optional, expert demos)
- Number of iterations N

Algorithm (DAgger)

1) Initialize policy π_1 by training on D_0

2) For $i = 1, \ldots, N$

- Roll out policy π_i to collect states $\{s_t\} \sim d_t^{\pi_i}$
- Query expert actions $a_t = \pi_E(s_t)$
- Aggregate dataset: $D_i = D_{i-1} \cup \{(s_t, \pi_E(s_t))\}$
- Train next policy:

$$\pi_{i+1} = \arg\min_{\pi} \mathbb{E}_{(s,a)\sim D_i}\left[-\log \pi(a \mid s)\right]$$

Output: Best policy among $\{\pi_i\}_{i=1}^N$

DAgger Guarantee (Why It Fixes BC)

Recall (Behavior Cloning)

$$\mathbb{E}[\#\text{errors}] = O(T^2\epsilon) \quad (\text{train on } d_t^{\pi_E},\ \text{test on } d_t^{\pi})$$

DAgger key property

- Training data drawn from the learner's induced distribution d^{π_i}
- No train–test distribution mismatch

Guarantee (informal)

- If the supervised learner has regret ϵ on the aggregated dataset, then

$$\mathbb{E}[\#\text{errors over } T] = O(T\epsilon)$$

Interpretation

- Linear (not quadratic) dependence on horizon
- Distribution shift eliminated by construction

Practical Limitations of DAgger

Assumptions behind DAgger

- Access to an expert policy π_E
- Ability to query expert on arbitrary states
- Safe execution of intermediate policies

Practical limitations

- Expert queries are expensive or unavailable
- Early policies may be unsafe to deploy
- Dataset size grows linearly with iterations

Consequence

- DAgger is theoretically clean, but often impractical at scale

Behavior Cloning vs DAgger (Algorithmic Comparison)

ASPECT	BEHAVIOR CLONING	DAGGER
Training states	d^{π_E}	d^{π_i}
Data collection	Fixed dataset	Iterative, interactive
Train–test mismatch	Yes covariate shift	No by construction
Error scaling	$O(T^2\varepsilon)$	$O(T\varepsilon)$
Supervision	Expert demos only	Expert queried on learner states
Practicality	Easy, scalable	Costly, expert-dependent

KEY TAKEAWAY

DAgger fixes BC by aligning the training distribution with the evaluation distribution.

Offline Reinforcement Learning

From Imitation Learning to Offline RL

Limitation of imitation learning

- Objective: match expert behavior
- No notion of optimality beyond the expert
- Assumption so far: expert policy π_E is near-optimal

In reality, demonstrations may be:

- Suboptimal
- Inconsistent
- Heterogeneous

New goal

*Improve over the behavior in the dataset using a **reward signal***

Constraint

- Dataset is fixed
- No environment interaction allowed

Offline Reinforcement Learning: Problem Formulation

- We are given an MDP with states, actions, transitions, and rewards
- Instead of collecting fresh data, we only observe a fixed dataset

$$\mathscr{D} = \{(s_i, a_i, r_i, s_i')\}_{i=1}^{N}$$

generated by some behavior policy μ

- **Goal:** learn a policy π that maximizes expected return

$$J(\pi) = \mathbb{E}_{\pi}\left[\sum_{t=1}^{T} \gamma^{t-1} r(s_t, a_t)\right]$$

- **Main challenge:** we must improve the policy **without any further environment interaction**

Why Naïve RL Fails Offline (Bellman Backup Issue)

Bellman optimality equation

$$Q^\star(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[\max_{a'} Q^\star(s',a') \right]$$

Offline setting

- We only observe transitions (s, a, s') for state-action pairs in the dataset support
- There is no data for unseen actions $a' \notin \mathrm{supp}(\mathscr{D}(s'))$

Core problem

$\max_{a'} Q(s',a')$ requires evaluating Q on unseen actions at the next state

Consequence

- Bellman backups extrapolate beyond the support of the data
- The maximization step can amplify estimation errors

Extrapolation Error in Offline RL

- **Function approximation**

$$Q_\theta \approx Q^\star, \qquad \text{trained only on } (s, a) \sim \mathscr{D}$$

- Extrapolation
  - For $(s, a) \notin \mathrm{supp}(\mathscr{D})$, the estimate $Q_\theta(s, a)$ is not grounded by data
  - So the value on unseen actions can be arbitrary

Extrapolation Error in Offline RL

- **Function approximation**

$$Q_\theta \approx Q^\star, \qquad \text{trained only on } (s,a) \sim \mathscr{D}$$

- Extrapolation
  - For $(s,a) \notin \mathrm{supp}(\mathscr{D})$, the estimate $Q_\theta(s,a)$ is not grounded by data
  - So the value on unseen actions can be arbitrary
- **Bellman update**

$$Q_\theta(s,a) \leftarrow r(s,a) + \gamma \max_{a'} Q_\theta(s',a')$$

- **Failure mode**
  - Some unseen next actions (s',a') get overestimated
  - The max operator prefers these inflated values
  - Bootstrapping then propagates the error backward through the dataset

Distribution Shift and Support Mismatch

- **Behavior policy**
  - The dataset is generated by a behavior policy $\mu(a \mid s)$
- **Target policy**
  - We want to learn a policy $\pi(a \mid s)$
- **Required support condition**

$$\mathrm{supp}\big(\pi(\,\cdot \mid s)\big) \subseteq \mathrm{supp}\big(\mu(\,\cdot \mid s)\big) \qquad \forall s$$

Distribution Shift and Support Mismatch

- **Behavior policy**
  - The dataset is generated by a behavior policy $\mu(a \mid s)$
- **Target policy**
  - We want to learn a policy $\pi(a \mid s)$
- **Required support condition**

$$\mathrm{supp}\big(\pi(\cdot \mid s)\big) \subseteq \mathrm{supp}\big(\mu(\cdot \mid s)\big) \qquad \forall s$$

- **What goes wrong in practice**
  - The learned policy may choose actions never taken by the behavior policy
  - Then the corresponding value $Q(s, a)$ must be estimated off-support
- **Consequence**
  - Policy improvement is pushed toward actions not covered by the dataset
  - Standard guarantees from online or ordinary off-policy RL no longer apply

The Offline Deadly Triad

1) Function approximation

$$Q_\theta \text{ is trained only on } (s, a) \sim \mathscr{D}$$

2) Bootstrapping

- Bellman updates rely on estimated future values:

$$Q_\theta(s, a) \leftarrow r(s, a) + \gamma \max_{a'} Q_\theta(s', a')$$

3) Distribution shift / support mismatch

- The learned policy typically differs from the behavior policy: $\pi \neq \mu$

- So it may choose actions outside the support of the dataset

The Offline Deadly Triad

1) Function approximation

$$Q_\theta \text{ is trained only on } (s, a) \sim \mathscr{D}$$

2) Bootstrapping

- Bellman updates rely on estimated future values:

$$Q_\theta(s, a) \leftarrow r(s, a) + \gamma \max_{a'} Q_\theta(s', a')$$

3) Distribution shift / support mismatch

- The learned policy typically differs from the behavior policy: $\pi \neq \mu$

- So it may choose actions outside the support of the dataset

Consequence

$$\text{unconstrained extrapolation} + \max + \text{bootstrapping} \implies \text{unstable learning}$$

Key message: Offline RL failure is structural, not tied to one particular algorithm

Fundamental Limits: Identifiability in Offline RL

Offline data constraint

$$\mathscr{D} \sim \mu, \qquad \mathrm{supp}(\mathscr{D}) \subseteq \mathscr{S} \times \mathscr{A}$$

Key question: Can the optimal policy $\pi^\star$ be identified from $\mathscr{D}$ alone?

Fundamental Limits: Identifiability in Offline RL

Offline data constraint

$$\mathscr{D} \sim \mu, \qquad \mathrm{supp}(\mathscr{D}) \subseteq \mathcal{S} \times \mathcal{A}$$

Key question: Can the optimal policy $\pi^\star$ be identified from $\mathscr{D}$ alone?

Negative result (informal)

- In general, no
- There can exist two different MDPs, M_1 and M_2, such that:
  - they agree on every (s, a) covered by the dataset
  - but they imply different optimal policies

- **Interpretation**
  - The data only tells us what happens on the support of $\mathscr{D}$
  - Outside that support, the environment may behave in different ways that are indistinguishable from the dataset

- **Conclusion**

Offline RL is not identifiable without additional assumptions

Non-Identifiability: Two-MDP Construction

- Given
  - Offline dataset $\mathscr{D}$ collected by behavior policy μ
  - For some state s, action a' never appears in $\mathscr{D}$: $(s, a') \notin \mathrm{supp}(\mathscr{D})$

Non-Identifiability: Two-MDP Construction

- Given
  - Offline dataset $\mathscr{D}$ collected by behavior policy μ
  - For some state s, action a' never appears in $\mathscr{D}$: $(s, a') \notin \mathrm{supp}(\mathscr{D})$
- Construct two MDPs M_1, M_2 :

$$\text{For all } (s,a) \in \mathrm{supp}(\mathscr{D}) : \; P_1(\,\cdot \mid s,a) = P_2(\,\cdot \mid s,a), \qquad r_1(s,a) = r_2(s,a)$$

Non-Identifiability: Two-MDP Construction

- Given
  - Offline dataset $\mathscr{D}$ collected by behavior policy μ
  - For some state s, action a' never appears in $\mathscr{D}$: $(s, a') \notin \mathrm{supp}(\mathscr{D})$
- Construct two MDPs M_1, M_2 :

$$\text{For all } (s,a) \in \mathrm{supp}(\mathscr{D}) : \; P_1(\cdot \mid s,a) = P_2(\cdot \mid s,a), \qquad r_1(s,a) = r_2(s,a)$$

- For unseen action a' at state s:

$$r_1(s,a') = +1, \qquad r_2(s,a') = -1$$

Non-Identifiability: Two-MDP Construction

- Given
  - Offline dataset $\mathscr{D}$ collected by behavior policy μ
  - For some state s, action a' never appears in $\mathscr{D}$: $(s, a') \notin \mathrm{supp}(\mathscr{D})$
- Construct two MDPs M_1, M_2 :

$$\text{For all } (s,a) \in \mathrm{supp}(\mathscr{D}) : \; P_1(\cdot \mid s,a) = P_2(\cdot \mid s,a), \qquad r_1(s,a) = r_2(s,a)$$

- For unseen action a' at state s:

$$r_1(s,a') = +1, \qquad r_2(s,a') = -1$$

- Result
  - Both M_1, M_2 are consistent with $\mathscr{D}$
  - Optimal actions at s differ: $\pi^\star_{M_1}(s) = a', \qquad \pi^\star_{M_2}(s) \neq a'$

Non-Identifiability: Two-MDP Construction

- Given
  - Offline dataset $\mathscr{D}$ collected by behavior policy μ
  - For some state s, action a' never appears in $\mathscr{D}$: $(s, a') \notin \mathrm{supp}(\mathscr{D})$
- Construct two MDPs M_1, M_2 :

$$\text{For all } (s,a) \in \mathrm{supp}(\mathscr{D}) : \; P_1(\,\cdot \mid s,a) = P_2(\,\cdot \mid s,a), \qquad r_1(s,a) = r_2(s,a)$$

- For unseen action a' at state s:

$$r_1(s,a') = +1, \qquad r_2(s,a') = -1$$

- Result
  - Both M_1, M_2 are consistent with $\mathscr{D}$
  - Optimal actions at s differ: $\pi^\star_{M_1}(s) = a', \qquad \pi^\star_{M_2}(s) \neq a'$
- Conclusion: $\pi^\star$ **is not identifiable from** $\mathscr{D}$

What Assumptions Make Offline RL Possible?

- Non-identifiability implies Offline RL requires additional assumptions

What Assumptions Make Offline RL Possible?

- Non-identifiability implies Offline RL requires additional assumptions

Common classes of assumptions

1) Coverage / support assumptions

$$\mathrm{supp}(\pi(\cdot \mid s)) \subseteq \mathrm{supp}(\mu(\cdot \mid s)) \qquad \forall s$$

2) Smoothness / generalization

- $Q^\star(s, a)$ varies smoothly over (s, a)
- Function class extrapolates correctly

3) Policy restriction

- Constrain π to stay close to μ
- Avoid unseen actions

Key implication: No assumption $\Rightarrow$ No guarantees

Conservative Offline RL: Core Principle

- Problem recap
  - Optimal actions off-support are not identifiable
  - Extrapolation errors are amplified by policy improvement
- Core principle: Avoid optimistic estimates outside the dataset support
- Conservative learning objective
  - Prefer policies with lower but reliable value estimates
  - Penalize uncertainty or deviation from data
- Design goal

$$\max_{\pi} J(\pi) \qquad \text{s.t.} \qquad \pi \text{ stays within regions supported by } \mathscr{D}$$

Conservative Value Estimation

- Standard Q-learning objective

$$\min_{\theta} \ \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[\left(Q_\theta(s,a) - (r + \gamma \max_{a'} Q_\theta(s',a')) \right)^2 \right]$$

- Problem
  - The maximization over a' may select actions not supported by the dataset
  - This can produce systematic overestimation

Conservative Value Estimation

- Standard Q-learning objective

$$\min_{\theta} \ \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[\left(Q_\theta(s,a) - (r + \gamma \max_{a'} Q_\theta(s',a')) \right)^2 \right]$$

- Problem
  - The maximization over a' may select actions not supported by the dataset
  - This can produce systematic overestimation

- Conservative modification

$$\min_{\theta} \ \mathbb{E}_{\mathcal{D}}[\text{Bellman error}] \ + \ \alpha \, \mathbb{E}_{s\sim\mathcal{D}} \left[\log \sum_{a} \exp(Q_\theta(s,a)) - \mathbb{E}_{a\sim\mu(\cdot|s)} Q_\theta(s,a) \right]$$

- Effect
  - Penalizes large Q-values on actions not well supported by the data
  - Encourages conservative value estimates off-support

Policy Regularization: Staying Close to the Dataset

- **Problem**
  - Policy improvement favors actions with high estimated Q-values
  - But those actions may lie outside the support of the dataset
- **Regularized policy optimization**

$$\max_{\pi} \; \mathbb{E}_{s\sim\mathcal{D}}\Big[\mathbb{E}_{a\sim\pi(\cdot|s)}Q(s,a)\Big] \; - \; \lambda\,\mathbb{E}_{s\sim\mathcal{D}}\Big[\mathrm{KL}\big(\pi(\,\cdot\mid s)\,\|\,\mu(\,\cdot\mid s)\big)\Big]$$

- **Interpretation**
  - Increase expected value under the learned policy
  - Penalize deviation from the behavior policy
- **Effect**
  - Keeps π close to actions supported by the dataset
  - Reduces off-support policy improvement

Bias–Performance Tradeoff in Offline RL

- Penalizing unseen actions lowers estimated values
- Regularization also restricts how far the policy can improve

- **Tradeoff**

$$\textit{more conservatism} \implies \textit{lower variance, higher bias}$$

$$\textit{less conservatism} \implies \textit{higher variance, greater risk of overestimation}$$

- **Key implication**
  - Offline RL methods differ mainly in how they choose this bias-variance tradeoff
  - The central question is how conservative to be, and where

Summary

Conceptual Arc: From BC to Offline RL

- In behavior cloning, the data distribution is controlled by the expert
- In DAgger, the learner can influence the data distribution through interaction and relabeling
- In offline RL, the dataset is fixed, so the learner has no control over coverage
- That is why offline RL is the hardest setting: it asks for policy improvement without the ability to correct the data distribution

BC $\longrightarrow$ DAgger $\longrightarrow$ Offline RL

Core Difficulties in Learning from Data

Shared challenges in BC, DAgger, and offline RL

1) Distribution shift

$$d^{\text{train}} \neq d^{\text{test}}$$

2) Support mismatch

$$\text{supp}(\pi) \nsubseteq \text{supp}(\mathscr{D})$$

3) Extrapolation error

- Value estimates are unconstrained off-support
- Errors are amplified by maximization and bootstrapping

Key distinction

- In online RL, distribution shift can be corrected through interaction
- In offline learning, distribution shift is permanent because no new data can be collected

Take-Home Messages

From imitation learning to offline RL

- **Imitation learning**
  - Reduces sequential decision-making to supervised learning
  - Fails under distribution shift
  - Compounding error can scale as $O(T^2\varepsilon)$
- **DAgger**
  - Addresses distribution shift through interactive data collection
  - Improves the error dependence to $O(T\varepsilon)$
  - Often impractical, but conceptually important
- **Offline RL**
  - Reintroduces rewards while keeping the dataset fixed
  - Suffers from support mismatch and extrapolation
  - Is ill-posed without additional assumptions
- **Core principle**

Reliable learning from fixed data requires conservatism

Recommended Reading

- Foundations
  - Sutton & Barto (2nd ed.): Ch. 11 — Off-policy methods with approximation
    (deadly triad, divergence, support issues)

- Imitation Learning
  - Ross, Gordon, Bagnell (2011)
    A Reduction of Imitation Learning to No-Regret Online Learning
    (BC compounding error, DAgger guarantees)

- Offline Reinforcement Learning
  - Levine et al. (2020)
    Offline Reinforcement Learning: Tutorial, Review, and Perspectives
    (support mismatch, extrapolation, limitations)

- Conservative Offline RL
  - Kumar et al. (2020)
    Conservative Q-Learning (CQL)
    (pessimism in value estimation)